

Table of Contents

Basics of Java programming.....	2
Language fundamentals.....	2
Operators and assignments.....	4
Declarations and Access Control.....	7
Control flow, exception handling and assertions.....	9
Object-oriented programming.....	10
Nested classes and Interfaces.....	13
Object lifetime.....	14
Threads.....	15
Fundamental Classes.....	16
Collections and Maps.....	16

This work is primarily intended to be a guideline for those taking the **Sun Certified Programmer for the Java 2 Platform 1.4** examination. It is recommended that the candidate check Sun's website for any changes in the pattern of the exam.

Copyright © 2005 Rohit George.
Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled "GNU Free Documentation License".

BASICS OF JAVA PROGRAMMING

- Objects are manipulated through object references (and aliases). new operator returns a reference to a new instance of the class instantiated.
- Objects have their copy of instance members, while static members belong only to the class. Local (or Automatic) variables are created inside methods/ blocks.
- Static members may be accessed by the class name as well as the object references, but Instance variables can only be accessed by the object reference.
- Two fundamental mechanisms of building new classes from existing ones :
 - Inheritance (IS-A relationship)
 - Aggregation (HAS-A relationship)
- Java supports aggregation where only the **references** to objects and **values** of primitive data types are stored within the aggregation class.
- Java 2 SDK enforces that at the most only one class in the source file has public accessibility. Each class definition in a source file is compiled into a separate class file, containing Java byte code. There can be any number of classes named 'main' created in a class, provided they take other arguments, are not public and static, or return a value.

LANGUAGE FUNDAMENTALS

- Identifier is composed of a sequence of characters, where each character could be either a letter, currency symbol, digit or connecting punctuation. However first character can't be a digit.
Examples of illegal identifiers : 48chevy, all@hands, grand-sum
- Keywords :

abstract	default	implements	protected	throw
assert	do	import	public	throws
boolean	double	instanceof	return	transient
break	else	int	short	try
byte	extends	interface	static	void
case	final	long	strictfp	volatile
catch	finally	native	super	while
char	float	new	switch	
class	for	package	synchronize	
continue	if	private	This	

- Reserved Literals :

null	true	false
------	------	-------

- Reserved Keywords not currently in use :

const	goto
-------	------

- synchronized, abstract, strictfp and native can only be applied to methods. transient and volatile can only be applied to variables. static and final can be applied to both.
- The class containing at least one abstract method has to be declared as abstract. However an abstract class need not have any abstract methods in it.

- Literal denotes a constant values :

- Integer Literals int, long, byte, short, octal, hexadecimal
- Floating point Literals suffix f or F, d or D; exponent notation 1.2E-2
- Boolean Literals true or false
- Character Literals single quotes, hexadecimal with prefix \u, \ddd where ddd is octal
- String Literals double quotes

- Comment are of three types :

- single line // ... to end of line
- multiple line /* ... */
- javadoc comment /** ... */

- Comment start sequences are ignored when occurring inside comments, hence making nesting of comments impossible.

- Primitive Data Types

byte	8 bits	-2^7	$+2^7-1$	Boolean
short	16 bits	-2^{15}	$+2^{15}-1$	Short
int	32 bits	-2^{31}	$+2^{31}-1$	Integer
long	64 bits	-2^{63}	$+2^{63}-1$	Long
char	16 bits	0x0	0xffff	Character
float	32 bits			Float
double	64 bits			Double
boolean		true or false only		Boolean

- Default values for fields are false, '\u0000', 0L, 0, 0.0F, 0.0D and null. All static, instance variables as well as object references are automatically initialized. Local variables and object references need to be explicitly initialized before use.

- If a reference is set to null, then the program will compile but throw a `NullPointerException` at runtime.

OPERATORS AND ASSIGNMENTS

- All binary operators except for relational and assignment operators associate from left to right. Relational operators are non-associative.
All unary operators (except for unary postfix increment and decrement), all assignment operators, and the ternary conditional operator operate from right to left.
- **Precedence** rules are used to determine which operator to apply first if there are two operators with *different* precedence, following each other in the expression.
Associativity rules are used to determine which operator to apply first if there are two operators with the *same* precedence, following each other in the expression.

Postfix	<code>[] . (parameters) (expr)++ (expr)--</code>
Unary prefix	<code>++(expr) --(expr) +(expr) -(expr)</code>
Unary prefix creation and cast	<code>new (type)</code>
Multiplicative	<code>* / %</code>
Additive	<code>+ -</code>
Shift	<code><< >> >>></code>
Relational	<code>< <= > >= instanceof</code>
Equality	<code>== !=</code>
Bitwise/logical AND	<code>&</code>
Bitwise/logical XOR	<code>^</code>
Bitwise/logical OR	<code> </code>
Conditional AND	<code>&&</code>
Conditional OR	<code> </code>
conditional	<code>?:</code>
Assignment	<code>= += -= *= /= %= <<= >>= >>>= &= ^= =</code>

- Java guarantees that *all* operands of an operator are fully evaluated *before* the operator is applied. The only exceptions are `&&`, `||` and `?:`. Also the operands of the operators are evaluated from left to right. For example, in the case of binary operators, if the evaluation of the left hand operator causes an exception, then the

right hand expression is not evaluated.

- Casting between primitives and references is not permitted. Boolean values cannot be cast to other data values and vice versa. `null` can be cast to any reference type.
- All conversions between `byte`, `short` to `char` are considered narrowing, because conversion to the unsigned `char` can lead to loss in information.
- Numeric Promotions
 - Unary Numeric Promotion : the operator is promoted to `int` if lesser than `int`.
Examples : operands of `+`, `-`
operand of `~`
during array creation, value in `[ ]`
indexing array elements, value in `[ ]`
operands of `<<`, `>>`, `>>>`
 - Binary Numeric Promotion : both operands are converted to the broadest of the two and to `int` at the least.
Examples : operands of `*`, `/`, `%`, `+`, `-`
operands of `<`, `<=`, `>`, `>=`
operands of `==`, `!=`
operands of `&`, `^`, `|`
- Type Conversion can occur in the following contexts :
 - Assignments involving primitive data types and reference types.
 - Method invocation involving parameters of primitive data types.
 - Arithmetic expression evaluation involving numeric types.
 - String concatenation involving objects of `String` and other data types
- The assignment statement is an expression statement, which returns the value of the expression on the right hand side.
- Integer values when widened to floating point values may lead to loss of precision.
- Implicit Narrowing Primitive Conversions take place when all of the following are true:
 - source is a constant expression of type `byte`, `short`, `char` or `int`.¹
 - destination is `byte`, `short` or `char` type.

¹ That means the variable must be declared `final` for implicit narrowing to take place.

- Value of source is determined to be in the range of the destination.
- Integer division by 0 and remainder by 0 gives Arithmetic Exception.
- Floating point errors throw NaN and Infinity, -Infinity.
- `(-0.0==0.0)` is true
`(0.0/0.0)` is NaN
any operation involving NaN gives NaN
any comparison involving NaN (except `!=`) gives NaN
square root of negatives gives NaN
to check if value is NaN, use `isNaN()`, defined in `java.lang.Float` and `java.lang.Double`.
- In the case of Arithmetic Compound Assignment Operators (`*=`, `/=`, `%=`, `+=`, `-=`), since they have the **lowest** precedence of all operators in Java, the full expression on the right will always be evaluated before the assignment. Also the left hand side variable (also in the expansion on the right side) is evaluated only *once*.
- Prefixed Increment/Decrement Operator first performs operation, then uses new value as result of the expression. Postfixed Increment/Decrement Operators use the current value as the result of the expression first and then perform the operation.
- Increment and Decrement operators cannot be associated, like in `(++(++a))` since the inner increment does not result in a *variable* for the other increment to operate on. Similarly, Relational and Equality operators are not associative since first operation would result in a boolean which cannot be the operand of the second.
- In the case of Object References, the Equality and the Inequality operators check whether the two references denote the same object or not. The state of the objects is not compared. To check for Object Value Equality, `equals()` method of class `Object` may be used, provided the class overrides the method. Otherwise Object Reference Equality and Object Value Equality are the same.²
- Boolean Logical Operators can be applied to boolean operands to return a boolean value, or to integral operands to perform bitwise operations. But Conditional Operators can only be applied to boolean operands.
- Adding 1 to the 1's complement of a positive binary integer gives its negative. And vice-versa. Hence if given a binary number and its signed bit is one, to find the decimal equivalent, we need to take the 2's complement, determine the decimal value and then affix a negative sign to it.

² `String`, `Date` and wrapper classes override `equals()`. `StringBuffer` does not.

- For the shift operator, a bit mask value of 0x1F (31, for int) or 0x3F (63, for long) is used by AND-ing with the shift distance value to ensure that the shift distance always stays in the range. (result of masking is <shiftvalue>%32 or 64).
- The conditional operator can be nested. It is associative from right to left.
- Formal parameters are those defined in the method definition. Actual parameters are those passed during the method call (which may vary from call to call).
 - The number of Actual parameters must equal the number of Formal parameters.
 - Individual Formal and Actual parameter pairs must be type-compatible.
- For parameter passing there are **no** implicit narrowing conversions.
- If an object reference is declared as final then it means that the reference value cannot be changed, however state of the object can be changed as usual.

DECLARATIONS AND ACCESS CONTROL

- Array Declarations


```
char[] a;
char a[];
```
- Array Constructions


```
a = new char[5];
```
- Combined Declaration and Construction


```
<element type1>[]<array name>=new <element type2>[<array size>];3
```
- Combined Declaration, Construction and explicit Initialization


```
<element type>[]<array name>={<array initialize list>};4
char[] a = {'b','c','d','e'};
```
- Anonymous Array Construction


```
new char[]{'b','c','d','e'}
```
- In the case of multidimensional arrays, you may specify the size only for the first dimension:


```
String[][] s = new String[3][];
```
- Accessing an incorrect index in the array results in an `ArrayIndexOutOfBoundsException`.

³ Where `element type 2` must be assignable to `type 1`.

⁴ The list cannot contain the array name.

- A class has two components :
 - Static Context : contains static methods, static field initializers and static initializer blocks.
 - Non-Static Context : has instance methods, constructors, non-static field initializers and instance initializer blocks.
- The *signature* of a method contains only the method name and the formal parameter list.
- All instance methods are passed an implicit reference to the *current object*, which can be referenced in the body of the instance by the keyword `this`, which can be used in any non-static context.⁵
- When a subclass object is created, all the superclass constructors are invoked in order starting from the top of the hierarchy.
- Constructors cannot return a value and hence **cannot** specify a return type, not even `void`.
- The only action taken by the default constructor is to call the superclass constructor.
- If any explicit constructors are defined, then implicit default constructor will not be provided. It must be explicitly implemented.
- Static code cannot access non-static members. Also `this` and `super` are unavailable. However, the Non-Static code can refer to the static context members.
- An interface is an implicitly `public`, `abstract` class containing implicitly `static`, `final` variables and `abstract` methods (if any). Interfaces with empty bodies are used as markers to tag classes as having a certain property. Such are also called *ability* interfaces.⁶

CONTROL FLOW, EXCEPTION HANDLING AND ASSERTIONS

- The type of expression of a switch statement must be non-long integral. All labels in a switch construct including the default label are optional. But there cannot be more than one default label. The case label values must be in range of type of switch expression.
- Switch blocks may be nested and the case labels can be redefined inside them.

⁵ Since the static context is not executed in the context of any object.

⁶ For example `java.lang.Cloneable`, `java.io.Serializable`, `java.util.EventListener`

- In the for loop header, two semicolons are mandatory. The 'crab' (; ;) is commonly used to construct infinite loops. If in the for loop, pre-increment instead of post-increment is used, it makes no difference to the behavior of the loop.
- The scope of a label is the entire statement prefixed by the label. So it can't be redeclared as a label inside the statement. A statement may have multiple labels. A declaration statement **cannot** have a label.
- `break()` terminates the loop or block it is in, whereas `continue()` skips that particular iteration of the loop and continues with the next one.
- All exceptions are derived from `java.lang.Throwable` class and have the methods `getMessage()` and `printStackTrace()`. `RuntimeException`, `Error` and their derivatives are called unchecked exceptions. All others are checked⁷. In particular `Asserts` throw unchecked `AssertionError`.
- In a try-catch-finally construct, block notation is mandatory. There can be zero or more catch blocks but a maximum of one finally block. The catch and finally blocks must always appear in conjunction with the try and in *that order*.
- After catching an exception, the control continues from the class which handled the exception. If the finally block throws an exception, then this is propagated regardless of how the try block or any catch block were executed. Also, the new exception overrides any previously unhandled exception. A value returned by a finally block will supersede other return statements.
- Any exception propagated to a finally block is **nullified** by any return statement in the finally block.
- When a subclass is created it can override a superclass method, but the new throws clause can only throw a subset of the previous exceptions (including their subclasses)
- To compile assertions use `$ javac -source 1.4 programname.java` and during running, enable it by either `-enableassertions`, or `-ea` flags. Disable by `-disableassertions` or `-da`. These options affect all *non-system* classes only⁸. For enabling or disabling system class assertions, use `-enablesystemassertions`, `-esa` and `j -disablesystemassertion`, `-dsa`.

⁷ They inherit from `Exception`

⁸ System classes are those in `java.*` packages

- Private, overridden and hidden members of the superclass are not inherited.
- This ability of a superclass reference to denote objects of its own class and its subclasses at runtime is called **polymorphism** and is carried out by *dynamic method lookup*. A call to a private instance method is not polymorphic since such a call can only occur within the class and gets bound to the private method implementation at compile time.
- Overriding of Instance Methods involve defining a method in the subclass with the same method signature (which does not encompass final modifier for the parameters, but **does** include the return type of the method⁹). Note that the new method **cannot** narrow the accessibility but can *widen* it¹⁰, and also it can only specify all, none or a subset of the former's exceptions.
- An instance method in the subclass cannot override a static method, a final method and a private method in the superclass. In the first case it will result in **hiding** of the static method, in the second a compile time error, and in the third it will result in the creation of a completely separate method with identical signature.
- When an instance method is invoked on an object using a reference, it is the *class of the current* object denoted by the reference, not the type of reference, that determines which method is implemented.

When a field of an object is accessed using a reference, it is the *type of the reference*, not the class of the current object denoted by the reference, that determines which field will actually be accessed. (Treatment is just the same for static methods and variables too)¹¹

```
class Vehicle {
    String mytype = new String("Vehicle!");
    void sayType() {
        System.out.println("I am a vehicle");
    }
}
class Car extends Vehicle {
    String mytype = new String("Car!");
    void sayType() {
        System.out.println("I am a car");
    }
}
```

⁹ In the case of Overloading return type may vary.

¹⁰ Somewhat clashing with the subset ideal in exceptions.

¹¹ Instance members are **overridden**. Instance variables, static methods and static variables are **hidden**.

We are overriding the field and the method. This is how they behave :

```
Vehicle i = new Vehicle;  
System.out.println(i.mytype);    // prints- Vehicle!  
i.sayType();                     // prints- I am a vehicle  
  
Car j = new Car;  
System.out.println(j.mytype);    // prints- Car!  
j.sayType();                     // prints- I am a car  
  
Vehicle k = new Car;  
System.out.println(k.mytype);    // prints- Vehicle!  
k.sayType();                     // prints- I am a car
```

- Constructors cannot be inherited or overridden. They can be overloaded but only in the same class. Java requires that a `this()` or `super()` call in the constructor (if any) be the *first* statement in the constructor.
- If a superclass only defines non-default constructors, then its subclasses cannot rely on implicit `super()` call to be inserted. This will result in compile time error.
- A class choosing to partially implement the methods of an interface must be declared as `abstract`.
- Regardless of how many interfaces are implemented, only a single implementation is provided for a member even if it has multiple declarations in the various interfaces.
- Rules for assigning reference values (same rules apply for passing references)
 $A = B$, where A = Destination reference, B = Source reference
 - *B is a class type*
 A is superclass of B
 A is interface type that B implements
 - *B is interface type*
 A is `Object`
 A is superinterface of B
 - *B is an array type*
 A is `Object`
 A is array type, where element type of B is assignable to element type of A
- Casting of References checks that the reference of the source is assignable to the reference of the destination. The compile time check determines if both the references can denote objects of a reference type that is a common subtype of both. At runtime it is the type of the actual object denoted by the source reference that determines the outcome of the operation. Hence a `ClassCastException` may result.

- References of an interface type can denote objects of classes that implement this interface.

NESTED CLASSES AND INTERFACES

Entity	Declaration Context	Accessib- ility Modifiers	Enclosing Instance	Direct Access to Enclosing Context	Declarations in Entity Body
Top-Level Class (or Interface)	Package	Public or default	No	N/A	All that are valid in a class (or interface) body
Static Member Class (or Interface) ¹²	As static member of enclosing class or interface	All	No	Static members in enclosing context	All that are valid in a class (or interface) body
Non-static Member Class ¹³	As non-static member of enclosing class or interface	All	Yes	All members in enclosing context	Only non-static declarations + final static fields
Local Class ¹⁴	In block with non- static context	None	Yes	All members in enclosing context + final local variables	Only non-static declarations + final static fields
	In block with static context	None	No	Static members in enclosing context + final local variables	Only non-static declarations + final static fields
Anonymous Class	As expression in non-static context	None	Yes	All members in enclosing context + final local variables	Only non-static declarations + final static fields
	As expression in static context	None	No	Static members in enclosing context + final local variables	Only non-static declarations + final static fields

- ```

class TLC {
 static class SMC {}
 interface SMI {}
 class NSMC {}
 void nsm() {
 class NSLC {}
 }
 static void sm() {
 class SLC {}
 }
}

```

//Top Level Class

//Static Member Class

//Static Member Interface

//Non-static Member Class

//Non-static Local Class

//Static Local Class

<sup>12</sup> Can only be nested within other `static` members or top-level classes and interfaces.

<sup>13</sup> Cannot have `static` members.

<sup>14</sup> Cannot be specified with the keyword `static`.

```

 }
 SMC nsf = new SMC () {}; //Anonymous non-static
 static SMI sf = new SMI() {}; //Anonymous Static
}

```

- The expression `<enclosing class name>.this` evaluates to a reference that denotes the enclosing object (ie. of class `<enclosing class name>`) of current instance of non-static member class.
- In the Anonymous class, SMC is not the name of the Anonymous class but the name of the class that you are extending or the interface you are implementing.

## OBJECT LIFETIME

- The lifetime of an object is time between when it is created and garbage collected.
- Accessibility time of an object is time between when it is created and becomes unreachable.
- Resurrection after garbage collection can be achieved in the `finalize` block by assigning its `this` reference to a static field. But since a finalizer is called only once, an object can be resurrected just once.
- Eligibility for garbage collection
  - if reference variable of that object is set to `null` and no other reference is referring to it
  - if reference variable of that object is made to refer to some other object and there is no other reference referring to it
  - locally created objects after the method returns, provided they are not exported out of the method (ie. returned or thrown as exception)
- Since there is no guarantee that the garbage collector will ever run, there is also no guarantee that the finalizer will ever be called. Also no guarantee of the order in which the objects will be collected.
- `System.gc()` can be used to **request** garbage collection
- `System.runFinalization()` method can be called to **suggest** that any pending finalizers be run for the objects eligible for garbage collection.
- A Java application can use a Runtime object<sup>15</sup> to interact with the JVM. Some methods include
 

```

void gc()
void runFinalization()

```

---

<sup>15</sup> Obtained by calling `Runtime.getRuntime()`

```
long freeMemory()
long totalMemory()
```

- Static Initializer Blocks are primarily used to initialize static fields and consists of the keyword `static` followed by a local block. When a class is initialized, the initializer expression in static field declarations and static initializer blocks are executed in the order they are specified in the class.
- Instance initializer blocks can be used to initialize fields during object creation. It merely consists of a local block. The code in the block is executed every time an instance of the class is created.

## THREADS

- Defined by either extending `java.lang.Thread` or by implementing `java.lang.Runnable` interface. The `run()` method needs to be overridden. This method must be `public` and `void` and should take no arguments.

- Extending

```
class MyThread extends Thread {
 public void run() { /* ... */ }
}
```

You never call `run()` directly. Instead you call the `start()` method of the `Thread` class.

```
MyThread my = new MyThread();
mt.start();
```

- Implementing

```
class MyRunnable implements Runnable {
 public void run() { /* ... */ }
}
```

Here you construct a thread by passing the instance of the `Runnable` class as an argument.

```
MyRunnable mc = new MyRunnable();
Thread t = new Thread(mc);
t.start();
```

- Thread Constructors available

- `Thread()`
- `Thread(String name)`
- `Thread(Runnable runnable)`
- `Thread(Runnable runnable, String name)`
- `Thread(ThreadGroup g, Runnable runnable)`
- `Thread(ThreadGroup g, Runnable runnable, String name)`
- `Thread(ThreadGroup g, String name)`

- `wait()`, `notify` and `notifyAll()` have to be called from a synchronized context.
- A dead thread can never enter any other state, even if the `start()` method is called on it.
- `yield()` method causes the current thread to move from the running to the runnable state to give other threads a chance to run.
- When a thread calls `join()` on another thread, the currently running thread will wait until the thread it joins with has completed.
- The argument to a `synchronized` keyword (in the case of blocks) is the reference of the object to be synchronized on. In the case of methods, the `synchronized` keyword merely precedes the method.
- Invoking `start()` twice on the same thread will throw an `IllegalThreadStateException` at runtime.

## FUNDAMENTAL CLASSES

- The `Math` class is `final` and all methods defined in the `Math` class are `static`. This means you cannot inherit from the `Math` class and override the methods. Also, the `Math` class has a `private` constructor so you cannot instantiate it.
- `ceil()`, `floor()` and `random()` return double values. `round()` returns a long for double inputs and `int` for float inputs. Trigonometric methods take radians as arguments, so we use `Math.toRadians()`.
- All wrapper classes except `Character` have two constructors – one that takes the primitive value and another that takes the `String` representation of the value. `Character` takes `char` type as argument.
- Wrapper objects are immutable.
- All wrapper classes (except `Character`) define a static method `valueOf()` which returns the wrapper object corresponding to the primitive value represented by the `String` argument. Then `byteValue()`, `doubleValue()`, `floatValue()`, `intValue()`, `longValue()`, `shortValue()` are used to get the primitive values from the wrapper classes.
- Parser methods may be used to short-circuit the process. They throw `NumberFormatException`.

## COLLECTIONS AND MAPS

|             |         |           |               |               |
|-------------|---------|-----------|---------------|---------------|
| <b>Set</b>  | HashSet | TreeSet   | LinkedHashSet |               |
| <b>List</b> | Vector  | ArrayList | LinkedList    |               |
| <b>Map</b>  | HashMap | HashTable | TreeMap       | LinkedHashMap |

- Set cannot contain duplicate elements. Not ordered (implementation : HashSet)  
SortedSet holds elements in sorted ascending order. (implementation : TreeSet)  
**HashSet** is not ordered or sorted. (constant time performance for basic add and remove).  
**TreeSet** arranges the elements in ascending element order, sorted according to the natural order of the elements.  
**LinkedHashSet** is an ordered HashSet, which gives elements in the order of insertion or last access.
- List represents ordered collections which allow positional access and search and can contain duplicate elements. (implementation : LinkedList, ArrayList, Vector, Stack)  
**ArrayList** enables fast iteration and constant speed positional access.  
**Vector** is similar to ArrayList only slower since it is synchronized.  
**LinkedList** allows fast insertion and deletion at the beginning or end.  
Commonly used for implementing stacks and queues.  
**Stack** is subset of Vector, it is simple LIFO.
- Map matches unique keys to values. Each key maps to at most one value. It does not implement the Collection interface (implementation : HashMap, HashTable, WeakHashMap)  
SortedMap orders the mapping in the sorted order of keys.  
**HashMap** is not sorted or ordered. It allows one null key and many null values.  
**HashTable** is similar to HashMap but does not allow null keys and values.  
Also, it is slower because it is synchronized.  
**LinkedHashMap** iterated by insertion or last accessed order. Allows one null key and many null values.  
**TreeMap** is a map in ascending key order, sorted according to the natural order for the key's class.
- Two objects that are not equal can have the same hash code. The hash code of a null value is defined as zero.



- When you override equals() you should override hashCode()<sup>16</sup> but the reverse is not required.
- A BitSet is not part of the Collections Framework. It contains elements which are bits (ie. boolean primitive). Like Vector its not of fixed size and can grow as needed.

---

<sup>16</sup> Which returns `int`

## List of References

A thorough reading of the following is highly recommended, much of the info in the notes is from them.

- [1]     **“A Programmer's Guide to Java Certification, Second Edition”** - Khalid A. Mughal and Rolf W. Rasmussen.

*Pearson Education*

- [2]     **“Java Certification Success, Part 1 SCJP”**

*IBM developerWorks*

- [3]     **“Sun Certified Java Programmer Pre-Exam Essentials”** - Dylan Walsh

- [4]     **“Java Tips”** - Heather MacKenzie

## Disclaimers

No claims are made about the accuracy of this document and no responsibility is taken for any errors.

The original author cannot control, and may not be aware of, derived versions of the document.

Copyright © 2005 Rohit George.

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled “GNU Free Documentation License”.